

Contents

Audio Corpus Feature Preprocessor – User Manual	1
1. Overview	1
Architecture	2
Components and files	2
2. Installation	2
3. Pipeline Steps	3
Step 1: Missing Value Imputation	3
Step 2: Correlation-Based Feature Selection	4
Step 3: Log Transform	5
Step 4: Winsorization	6
4. Step Modes	6
5. Message Reference	7
Loading and state	7
Configuration	7
Execution	8
Outlet messages	8
6. Data Format	9
Input corpus (JSON)	9
Output corpus (JSON)	9
Transform parameters (JSON)	9
7. Visualization	10
The [v8ui] panel (<code>diagnostics_ui.js</code>)	10
The HTML report (jweb)	10
8. How to Read the Report Charts	11
9. Usage Workflows	12
Workflow A: Quick automatic processing	12
Workflow B: Step-by-step evaluation	12
Workflow C: Manual control with system guidance	12
Workflow D: Handling metadata columns	13
Workflow E: Comparing parameter settings	13
10. Integration with FluCoMa	14
Normalizing a new sound at runtime	14
11. Parameter Quick Reference	14
12. Troubleshooting	15

Audio Corpus Feature Preprocessor – User Manual

1. Overview

The Audio Corpus Feature Preprocessor is a tool for Max that prepares raw audio descriptor data for distance-based navigation, clustering, and nearest-neighbor queries. It sits between your feature extraction stage (e.g. FluCoMa `fluid.buf*` analyzers) and the corpus navigation stage (`fluid.kdtree~`, `fluid.grid~`).

Raw audio features are problematic for distance calculations because they have mismatched scales, skewed distributions, outliers, and redundancies. This tool applies a sequence of statistical transfor-

mations to fix these issues, producing a clean descriptor space where Euclidean distance reflects balanced perceptual similarity.

Architecture

The tool consists of three components:

- **preprocess_v5.js** – the main computation script, runs in a Max [v8] object
- **diagnostics_ui.js** – a visual dashboard, runs in a Max [v8ui] object
- **report_v3.html** – a rich diagnostic visualization, displayed in a [jweb] object

The preprocessing pipeline handles four steps. After these, the output is loaded into FluCoMa objects for the remaining normalization and spatial operations:

preprocess_v5.js		FluCoMa	
+-----+		+-----+	
Raw	1. Impute		5. Standardize
corpus ->	2. Correlate	-> JSON ->	or Robust
(JSON)	3. Log transform		6. UMAP
	4. Winsorize		7. KDTree
+-----+		+-----+	

Components and files

The tool comprises three user-facing components – **preprocess_v5.js**, **diagnostics_ui.js**, and **report_v3.html** – together with several supporting scripts that the patch loads internally. All of them are distributed inside the Max package. Because Max adds the whole package to its search path, they are found automatically regardless of how the package is organised internally, so the layout can change freely as files are added.

The pipeline reads and writes plain JSON files (in **fluid.dataset~** format). The generated files – the processed corpus, the transform parameters, and the diagnostics report data – are written to the locations you specify at commit and report time.

2. Installation

Copy the package folder into **Documents/Max 9/Packages/**, and install the **FluCoMa** and **Bach/EARS** packages from the Max Package Manager if they are not already present. All patches and scripts are then found automatically on Max's search path, wherever they sit inside the package, so no manual path configuration is required.

Create the objects in your Max patch:

- [v8 preprocess_v5.js @embed 1] -> the processing engine
- [v8ui @filename diagnostics_ui.js @size 620 520] -> the visual dashboard
- [jweb @size 800 900] -> the HTML report (loads report_v3.html)

Wire the outlets:

Outlet	Content	Connect to
0	Status messages	[route loaded mode committed error preview_done report_ready]
1	Diagnostics dict name	[v8ui] inlet
2	Bang (after commit)	[fluid.dataset~] load message

3. Pipeline Steps

Step 1: Missing Value Imputation

Purpose: Fill gaps in your feature data. Some audio analyzers produce NaN (Not a Number) when they cannot extract a meaningful value – for example, a pitch tracker applied to noise, or a rhythmic density estimator applied to a sustained drone. These NaN values will corrupt all downstream calculations.

What it does: - Scans every feature column for missing values (NaN, null, undefined) - Optionally treats a specific numeric value as a missing-value sentinel (e.g. 0 or -1) - Replaces each missing value with the median of that feature across all valid samples - Reports how many values were filled per feature and what percentage of the corpus was affected

When to use: Always enable this as a first pass, at least in auto mode. Even if you believe your data has no missing values, the detection costs nothing and catches hidden problems (like silent zeros that should have been NaN).

Parameters:

Parameter	Type	Default	Range	Description
<code>nan_sentinel</code>	float	none	any number	Treat this value as missing. Use 0 if your analyzer writes 0 for failed extractions, or -1 if it uses -1 as a flag
<code>fill_values</code>	float array	auto (median)	any numbers	Manual fill values, one per feature. In auto mode, the median is used

Example:

```
set_mode impute auto
set_param impute nan_sentinel 0    <- treat zeros as missing
```

Step 2: Correlation-Based Feature Selection

Purpose: Remove redundant features that carry overlapping information. If Brightness and Spectral Centroid are correlated at $r = 0.91$, keeping both means that “brightness-like” information gets double-counted in every distance calculation, distorting the balance of your descriptor space.

What it does: - Computes the Pearson correlation matrix across all features - Identifies every pair whose absolute correlation exceeds the threshold - For each correlated pair, determines which feature is more redundant overall (higher average correlation with all other features) - In auto mode: drops the more redundant feature from each pair using a greedy iterative algorithm - In manual mode: reports suggestions and waits for you to specify which features to keep

When to use: Enable when your feature set includes potentially overlapping descriptors. Disable if you’ve already hand-picked orthogonal features or if you need every feature for a specific analytical reason.

Parameters:

Parameter	Type	Default	Range	Description
threshold	float	0.85	0.0 - 1.0	Absolute correlation above which a pair is flagged. Lower = more aggressive pruning. 0.85 is conservative; 0.7 is aggressive
keep_indices	int array	none	original feature indices	Manual mode: explicitly list which original feature indices to keep. All others are dropped
prefer_indices	int array	none	original feature indices	Auto mode: when choosing between a correlated pair, prefer to keep these features

Example:

```
set_mode correlate auto
set_param correlate threshold 0.9      <- only drop very highly correlated pairs
set_param correlate prefer_indices 0 3  <- always prefer features 0 and 3
```

Example (manual – exclude metadata column):

```
set_mode correlate manual
set_param correlate keep_indices 0 1 2 3 4 5 6 7 8 9  <- drop column 10
```

Step 3: Log Transform

Purpose: Compress features with long-tailed (skewed) distributions. Audio features like Spectral Flux or Loudness Range often have most values clustered near zero with a few extreme values stretching the range by orders of magnitude. This compression prevents extreme values from dominating distance calculations and makes the distribution closer to symmetric.

What it does: - Applies the signed log1p transform: $y = \text{sign}(x) \times \log(1 + |x|)$ - This preserves the sign of negative values and handles zero gracefully ($\log1p(0) = 0$) - Compresses large values exponentially while barely affecting small values - Reports skewness before and after for each transformed feature

When to use: Enable for features that have visibly skewed distributions. In auto mode, the system detects skewness and applies the transform selectively. In manual mode, you choose which features to transform based on the system's suggestions.

Important note on probability-based systems: The log transform distorts probability densities via the Jacobian factor (e^y). If your system uses density-based similarity (rather than pure distance), be aware that rare extreme values will appear artificially dense in the transformed space. See the theoretical discussion in the project documentation for details.

Parameters:

Parameter	Type	Default	Range	Description
skew_threshold	float	2.0	0.0 - inf	Auto mode: apply log when skewness exceeds this value
indices	int array	none	original feature indices	Manual mode: apply log only to these features (uses original indices, automatically mapped to current columns after correlation pruning)

Example:

```
set_mode log auto
set_param log skew_threshold 1.5      <- transform anything moderately skewed
```

Example (manual):

```
set_mode log manual
preview                                <- see suggestions
set_param log indices 1 4              <- transform only features 1 and 4
```

Step 4: Winsorization

Purpose: Clip remaining outliers to the boundary percentiles. Even after log-transform, some extreme values may persist. Winsorization caps these extremes without removing any samples from the corpus – every sound remains addressable, but the most extreme values are pulled inward.

Named after: Charles P. Winsor, a biostatistician who proposed this technique in the 1940s.

What it does: - For each feature, computes the value at the lower percentile (default 2nd) and upper percentile (default 98th) - Clips all values below the lower boundary to that boundary, and all values above the upper boundary to the upper boundary - Reports how many values were clipped per feature and the boundary values

When to use: Almost always, unless you specifically need to preserve extreme values for your application. The default 2nd/98th percentile settings are very gentle – they only affect the outermost 4% of data.

Parameters:

Parameter	Type	Default	Range	Description
lo_pct	float	2	0 - 50	Lower percentile for clipping. 2 means clip below the 2nd percentile
hi_pct	float	98	50 - 100	Upper percentile for clipping. 98 means clip above the 98th percentile
boundaries	array of [lo, hi]	auto (computed)	any numbers	Manual mode: specify exact clip boundaries per feature instead of percentile-based

Example:

```
set_mode winsor auto      <- use default 2/98 percentiles
```

Example (aggressive clipping):

```
set_mode winsor auto
set_param winsor lo_pct 5
set_param winsor hi_pct 95      <- clip the outer 10% of data
```

4. Step Modes

Every step operates in one of three modes:

Mode	Behavior	Use when
<code>off</code>	Step is skipped entirely	You don't need this processing
<code>auto</code>	Step analyzes the data and applies its transform using internal heuristics. Reports what it did	You want quick results and trust the defaults
<code>manual</code>	Step analyzes the data and produces suggestions, but only applies transforms if you've explicitly set parameters	You want full control over what happens to your data

In manual mode, the workflow is: 1. Set the mode: `set_mode log manual` 2. Run preview: `preview` 3. Read the suggestions in the Max console 4. Set your parameters: `set_param log indices 1 4` 5. Preview again: `preview` 6. Repeat until satisfied, then `commit`

5. Message Reference

Loading and state

Message	Arguments	Description
<code>load</code>	<code><filepath></code>	Load a raw JSON corpus from a file. Accepts absolute paths or filenames in Max's search path
<code>load_dict</code>	<code>[dict_name]</code>	Load a raw JSON corpus from a named Max Dict (default name: <code>raw_dataset</code>). Recommended – fully Max-native
<code>reset</code>	<code>(none)</code>	Reset all modes to “off” and all parameters to defaults. Clears previous results
<code>get_modes</code>	<code>(none)</code>	Print current mode of each step to the Max console
<code>get_params</code>	<code><step></code>	Print current parameters of a step. Steps: <code>impute</code> , <code>correlate</code> , <code>log</code> , <code>winsor</code>

Configuration

Message	Arguments	Description
<code>set_mode</code>	<code><step></code> <code><off\ auto\ manual></code>	Set operating mode for a pipeline step

Message	Arguments	Description
<code>set_param</code>	<code><step> <key> <value...></code>	Set a parameter. Multiple values are space-separated

Execution

Message	Arguments	Description
<code>preview</code>	(none)	Run the full pipeline with current settings. Computes diagnostics, updates the [v8ui] panel, prints stats to Max console. Does NOT write files
<code>commit</code>	<code><corpus_path></code> <code>[params_path]</code>	Run pipeline and write output files. If <code>params_path</code> is omitted, it defaults to <code>corpus_path</code> with <code>_params.json</code> appended
<code>report</code>	<code>[json_path]</code>	Write report data to the <code>preprocess_report</code> Dict for the jweb report. Optionally also export a JSON file if a path is given. Must run <code>preview</code> first

Outlet messages

Outlet	Message	When
0	<code>loaded <n_samples></code> <code><n_features></code>	After successful load or <code>load_dict</code>
0	<code>mode <step> <mode></code>	After <code>set_mode</code>
0	<code>preview_done</code>	After preview completes
0	<code>committed <path></code>	After commit completes
0	<code>report_ready</code>	After report completes (route to <code>load_report</code> -> jweb)
0	<code>error <message></code>	On error
0	<code>reset</code>	After reset
1	<code>diagnostics <dict_name></code>	After preview (triggers [v8ui] redraw)
2	<code>bang</code>	After commit (trigger <code>fluid.dataset~ load</code>)

6. Data Format

Input corpus (JSON)

The input must be in `fluid.dataset~` format:

```
{
  "cols": 10,
  "data": {
    "sample_001": [1299.34, 10230.00, 48.50, ...],
    "sample_002": [2041.22, 8754.31, 12.87, ...]
  }
}
```

- `cols`: number of features (informational – the script uses actual array lengths)
- `data`: object mapping sample identifiers to feature arrays
- Optionally include `"feature_names": ["Brightness", "Loudness Range", ...]`

Output corpus (JSON)

Same format as input, but with transformed values and potentially fewer columns (if correlation pruning removed features):

```
{
  "cols": 8,
  "data": {
    "sample_001": [0.423, 1.872, 3.441, ...],
    "sample_002": [0.198, 2.103, 1.956, ...]
  }
}
```

Transform parameters (JSON)

Records every transform applied, for runtime use on new sounds:

```
{
  "steps_enabled": { "impute": true, "correlate": true, "log": true, "winsor": true },
  "modes": { "impute": "auto", "correlate": "manual", "log": "auto", "winsor": "auto" },
  "feature_names_original": ["Brightness", "Loudness Range", ...],
  "feature_names_output": ["Brightness", "Spectral Flux", ...],
  "kept_indices": [0, 1, 3, 4, 5, 7],
  "n_features_in": 10,
  "n_features_out": 8,
  "impute_medians": [3.2, 0.0, 47.1, ...],
  "impute_counts": [0, 12, 0, ...],
  "log_original_indices": [1, 4],
  "winsor_boundaries": [[0.12, 8.45], [0.03, 6.21], ...]
}
```

7. Visualization

The [v8ui] panel (`diagnostics_ui.js`)

A compact dashboard that updates automatically after each `preview`. Shows:

- **Header:** sample count, feature count (before -> after)
- **Mode indicators:** colored chips showing each step's mode (green = auto, orange = manual, grey = off)
- **Statistics table:** per-feature skewness and kurtosis, before vs after, with color coding (green = improved, red = worsened)
- **Correlation heatmap:** color-coded matrix of feature correlations (blue = negative, red = positive)
- **Suggestions panel:** compact list of system recommendations for manual-mode steps

Setup:

```
[v8ui @filename diagnostics_ui.js @size 620 520]
```

Connect outlet 1 of [v8 preprocess_v5.js] to the [v8ui] inlet.

The HTML report (`jweb`)

A rich, interactive visualization driven by the `report` command. The report data is written to a Max Dict named `preprocess_report`, and a static `report_v3.html` file reads it through the `jweb` bridge (`window.max.getDict`). Contains:

- **Pipeline flow diagram:** visual chain showing each step and what it did
- **Distribution histograms:** per-feature overlaid before/after histograms (40 bins)
- **Box plots:** all features side by side, before/after comparison
- **Scatter plots:** automatically generated for correlated pairs ($|r| > 0.5$)
- **Parallel coordinates:** sample subset showing multivariate structure
- **Interactive correlation heatmap:** mouse-hover shows exact r values
- **Stats table with sparklines:** inline mini-histograms next to each feature row
- **Suggestions:** detailed recommendation list

Setup:

```
[jweb @size 800 900]
```

Place `report_v3.html` in the same folder as your patch (or anywhere in the package).

Loading the report (recommended message-triggered wiring):

```
[v8 preprocess_v5.js] outlet 0  
-> [route report_ready]  
-> [message: load_report]  
-> [jweb]
```

The workflow is: run `preview`, then `report`. The [v8] object writes the Dict and outputs `report_ready` from outlet 0; routing that to a `load_report` message tells the HTML to read the Dict and render. A “Refresh” button also appears in the report if data isn’t ready on first load. Optionally add a `[dict preprocess_report]` object to your patch so the Dict stays visible and persistent.

8. How to Read the Report Charts

Each visualization answers a different question about the data. This section explains what each shows and what to look for.

Pipeline Flow. Shows the sequence of processing steps and what each did. Active steps are highlighted; “off” steps are dimmed. Read it left to right to see how the feature count and data changed from raw input to final output. Use it as a quick audit of which transforms were applied and their cumulative effect.

Distribution Histograms (before vs after). One histogram per feature, with red bars for the raw distribution and green bars after processing. The horizontal axis is the feature’s value; the vertical axis is how many sounds fall in each bin. This is the most informative single view in the report. What to look for: a red distribution bunched against one side with a long tail (skew) that becomes a more symmetric green shape means the log transform worked. A green distribution with sharp cliffs at the edges shows winsorization clipped the tails. A tall spike at a single value (often zero) may indicate hidden missing values masquerading as data. A distribution with two separate humps (bimodal) signals the feature may capture two distinct sound populations. If red and green are nearly identical, that step had little effect on this feature.

Box Plots (before vs after). A compact five-number summary per feature: the box spans the 25th to 75th percentile (the middle half of the data), the thick line is the median, and the whiskers reach the minimum and maximum. Red is before, green is after. What to look for: shortened whiskers after processing show outliers were pulled in by winsorization; a shifted median shows the log transform recentered the data; a box that shrinks shows reduced spread. Each feature has its own vertical scale because raw audio features live on very different ranges, so compare a feature’s before-vs-after pair, not one feature against another.

Correlated Pairs (scatter plots). Scatter plots for every feature pair whose absolute correlation exceeds 0.5, drawn from a sample of the corpus. Each dot is one sound; darker regions are where many sounds overlap, revealing where the data concentrates. What to look for: a tight diagonal cloud confirms a genuine linear relationship – the two features are redundant and are candidates for pruning. A curved or branched shape means the single correlation number understates a more complex dependency (and that linear decorrelation like PCA would not fully remove it). Distinct separate clusters warn that one correlation value is misleading because the relationship differs between groups. If this section reports “no pairs”, your features are already reasonably independent and feature selection has little to remove.

Parallel Coordinates. Each vertical line is one feature axis; each faint connecting line is a single sound traced across all of its feature values. This is the only view that shows multivariate structure – how all features relate simultaneously. What to look for: bundles of lines following the same path reveal groups of similar sounds (clusters); lines that cross sharply between two adjacent axes indicate those features are inversely related; an isolated strand far from the rest is an outlier sound. For corpora with many features and no clean clusters this view can look dense, but bundling and crossing patterns are still meaningful.

Correlation Matrix. A grid of every feature-pair correlation. Red cells are positive correlation, blue are negative, neutral cells are uncorrelated; the diagonal is always 1 (each feature with itself). Hover any cell for the exact value. What to look for: bright red or blue cells away from the diagonal

flag redundant feature pairs – these are what drive the correlation-pruning suggestions. A matrix that is mostly neutral means your features carry largely independent information, which is the desired state for balanced distance calculations.

Feature Statistics (table with sparklines). The numeric backbone, one row per feature. The sparkline is a miniature before/after histogram for a quick visual cue. Skewness measures asymmetry: 0 is symmetric, large positive means a long right tail, large negative a long left tail. Kurtosis measures tail heaviness: around 0 is normal-like, high values indicate heavy tails with outliers. Green numbers mean processing improved that statistic (moved it closer to zero); red means it worsened. What to look for: features still showing high skewness or kurtosis after processing may need a different transform, a manual log decision, or closer inspection in the histogram view.

Suggestions. Actionable recommendations generated while analyzing the data: which features to log-transform (with their skewness), which correlated pairs to consider dropping (with r values), how many missing values were found, and how many values winsorization would clip. In manual mode these are advisory – you decide whether to apply them via `set_param`.

9. Usage Workflows

Workflow A: Quick automatic processing

For rapid corpus preparation with sensible defaults.

```
load corpus_raw.json
set_mode impute auto
set_mode correlate auto
set_mode log auto
set_mode winsor auto
preview
commit corpus_processed.json
```

Workflow B: Step-by-step evaluation

Enable one step at a time to understand each transform's impact.

```
load corpus_raw.json
preview                                <- baseline statistics
set_mode impute auto
preview                                <- see imputation effect alone
set_mode log auto
preview                                <- see imputation + log effect
set_mode winsor auto
preview                                <- see all three combined
commit corpus_processed.json
```

Workflow C: Manual control with system guidance

Let the system analyze and suggest, but make all decisions yourself.

```
load corpus_raw.json
```

```

set_mode impute auto          <- auto is fine for imputation
set_mode correlate manual     <- want to control feature selection
set_mode log manual           <- want to control log transform
set_mode winsor auto          <- auto is fine for clipping
preview
;; Read suggestions in Max console:
;; [correlate] Brightness <-> Sp.Centroid r=0.91 -> suggest drop Sp.Centroid
;; [log] Loudness Range (orig 1): apply log1p (|skew| = 3.12 > 2.0)
;; [log] Spectral Flux (orig 4): apply log1p (|skew| = 4.21 > 2.0)
;; Apply your decisions:
set_param correlate keep_indices 0 1 3 4 5 6 7 <- drop feature 2 (Sp.Centroid)
set_param log indices 1 4 <- follow the log suggestion
preview <- verify your choices
commit corpus_processed.json
report <- write report Dict for jweb

```

Workflow D: Handling metadata columns

Your JSON has extra columns that aren't audio features (e.g. duration, sample rate).

```

load corpus_raw.json
;; Feature 10 has only 3 unique values -- it's metadata, not an audio feature
set_mode correlate manual
set_param correlate keep_indices 0 1 2 3 4 5 6 7 8 9
set_mode log auto
set_mode winsor auto
preview
commit corpus_processed.json

```

Workflow E: Comparing parameter settings

Try different thresholds to find the best setting.

```

load corpus_raw.json
set_mode correlate auto
set_mode log auto
set_mode winsor auto
;; Try conservative log threshold
set_param log skew_threshold 3.0
preview <- note results
;; Try moderate log threshold
set_param log skew_threshold 2.0
preview <- compare
;; Try aggressive log threshold
set_param log skew_threshold 1.0
preview <- compare
;; Pick the one that gave the best skewness reduction
set_param log skew_threshold 2.0
commit corpus_processed.json

```

10. Integration with FluCoMa

After committing, the output JSON is loaded into `fluid.dataset~`, then processed by FluCoMa objects:

```
[v8 preprocess_v5.js] outlet 2 (bang)
-> [message: load corpus_processed.json]
-> [fluid.dataset~ @name preprocessed]
-> [fluid.standardize~ @name scaler]      ;; or fluid.robustscale~
-> [fluid.dataset~ @name normalized]
-> [fluid.kdtree~ @name tree]

;; Optional: UMAP augmentation
[fluid.dataset~ @name normalized]
-> [fluid.umap~ @dimensions 3 @neighbors 50]
-> [fluid.dataset~ @name umap_coords]
```

Normalizing a new sound at runtime

When a new sound arrives that wasn't in the original corpus, apply the same transforms using the stored parameters:

1. Extract features from the new sound (same analyzer chain as the corpus)
2. Apply imputation: replace any NaN with the stored medians
3. Select features: keep only the indices listed in `kept_indices`
4. Apply log: for features in `log_original_indices`, compute $\text{sign}(x) \times \log(1 + |x|)$
5. Apply winsorization: clip to the stored `winsor_boundaries`
6. Apply the FluCoMa scaler: use `fluid.standardize~` transform message

The transform parameters file contains all the values needed for steps 2-5.

11. Parameter Quick Reference

Impute

Parameter	Default	Type	Description
<code>nan_sentinel</code>	<code>none</code>	float	Treat this value as NaN
<code>fill_values</code>	<code>median</code>	float array	Manual fill values per feature

Correlate

Parameter	Default	Type	Description
<code>threshold</code>	0.85	float (0-1)	Correlation threshold for pruning
<code>keep_indices</code>	all	int array	Manual: which original indices to keep
<code>prefer_indices</code>	none	int array	Auto: prefer keeping these features

Log

Parameter	Default	Type	Description
<code>skew_threshold</code>	2.0	float (≥ 0)	Auto: transform when $ \text{skewness} $ exceeds this value
<code>indices</code>	auto	int array	Manual: which original indices to log-transform

Winsor

Parameter	Default	Type	Description
<code>lo_pct</code>	2	float (0-50)	Lower clip percentile
<code>hi_pct</code>	98	float (50-100)	Upper clip percentile
<code>boundaries</code>	auto	array of [lo, hi]	Manual: exact clip boundaries per feature

12. Troubleshooting

“Error: cannot open file” - Make sure the file path is correct and the file exists - Try using just the filename (without the full path) if the file is in the same folder as your patch - Check that the file is valid JSON

“Error: run ‘preview’ first” - The `commit` and `report` commands require a `preview` to have been run first, so the pipeline results are available

No features transformed (log step) - In auto mode, check `skew_threshold` – it might be too high. Try `set_param log skew_threshold 1.5` - In manual mode, make sure you’ve set the indices: `set_param log indices 1 4`

Too many features dropped (correlate step) - Raise the threshold: `set_param correlate threshold 0.95` - Use `prefer_indices` to protect specific features - Switch to manual mode for full control

HTML report not generated - Check the Max console for error messages after running `report` - Click the Refresh button in the report, or check that `report_v3.html` is loaded in jweb - Make sure the output directory is writable

[v8ui] panel shows “No diagnostics” - Run preview first – the panel only updates after preview
- Check that outlet 1 of [v8 preprocess_v5.js] is connected to the [v8ui] inlet - Send bang to the [v8ui] to force a refresh